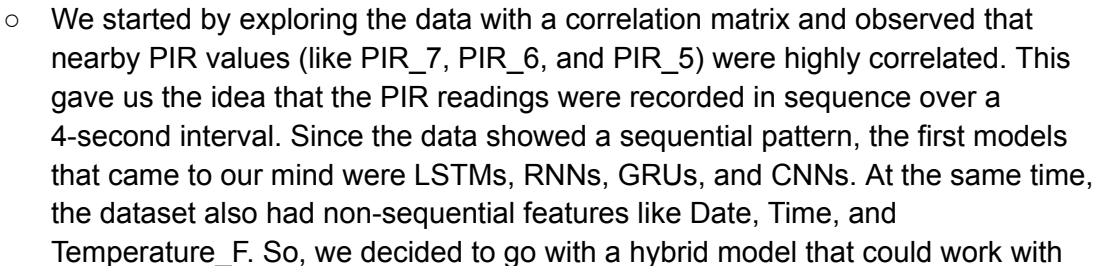


1. Problem Understanding:

- **Correlation matrix:**



both the sequential sensor data and the tabular metadata for classifying activities.

- **Challenges:**

- **Data Imbalance:** The distribution of activity labels is uneven, requiring the computation of class weights for balanced model training.
- **Multimodal Data:** The dataset comprises both tabular metadata and sequential PIR sensor readings, requiring distinct preprocessing workflows and tailored model architectures for each data type.
- **Temporal Patterns:** Effectively recognizing the temporal dynamics within the sensor readings is crucial for accurate activity classification, which calls for time-aware feature engineering and sequence modeling techniques.

- **Objective:**

- The aim is to build a reliable classification model that uses both the PIR sensor data and the metadata. The model is tested using 5-fold cross-validation, and its performance is measured using the average accuracy, standard deviation, and F1 scores.

- **Summary:**

- The dataset includes time series data from a PIR sensor along with some metadata. The approach starts with thorough preprocessing, such as scaling features and transforming time-related data. A hybrid model is then built to handle both types of data, using different scaling methods for the PIR readings and the metadata. The model's performance is tested using 5-fold cross-validation to make sure the predictions of activity labels are consistent and reliable.

2. Exploratory Data Analysis (EDA)

- **Data Loading:**

The dataset is loaded using `pandas.read_csv()`. We checked its shape, structure, and data types using `df.shape`, `df.info()`, `df.duplicated()` and `df.describe()`. To check data quality, we used `df.isnull().sum()` to look for missing values and examined unique counts for each column.

- **Data Cleaning and Preprocessing:**

- **Handling Missing Values and Duplicates:**

The dataset doesn't contain any missing values or duplicate rows, so no cleaning was needed in this part.

- **Time Feature Engineering:**

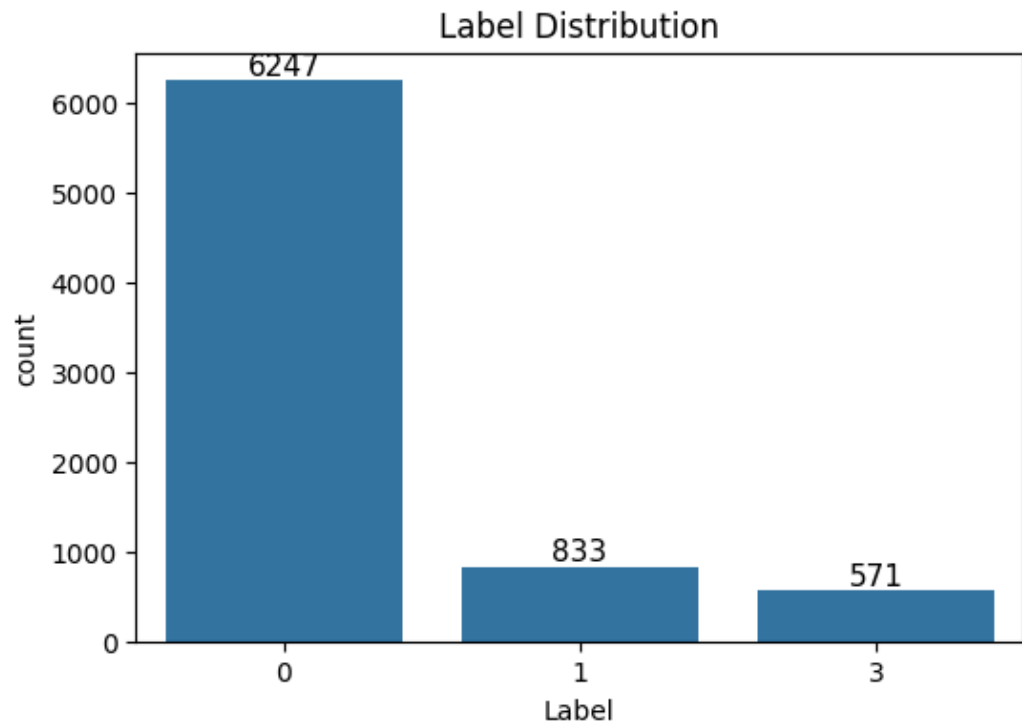
The Date and Time columns were merged into a single timestamp. From this, we extracted useful time-based features like hour, minute, second, day of the week,

and whether it was a weekend. To capture the cyclical nature of time, sine and cosine transformations were applied to the hour, minute, and second features.

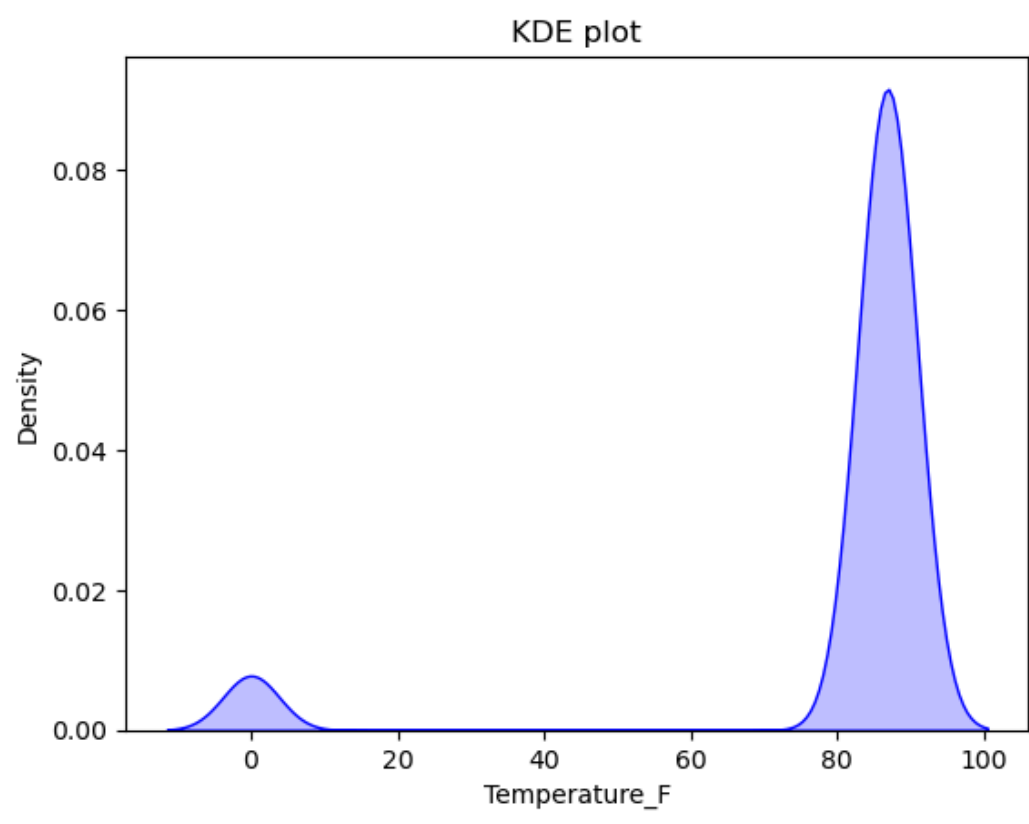
- **Visualization:**

We used kernel density plots (`sns.kdeplot`) to visualize the distribution of the Date (after converting it to ordinal format) and the Label. We also printed frequency counts and distributions for features like Temperature, Date, and one of the PIR (we chose PIR_10) to better understand the spread and variation in the data.

-



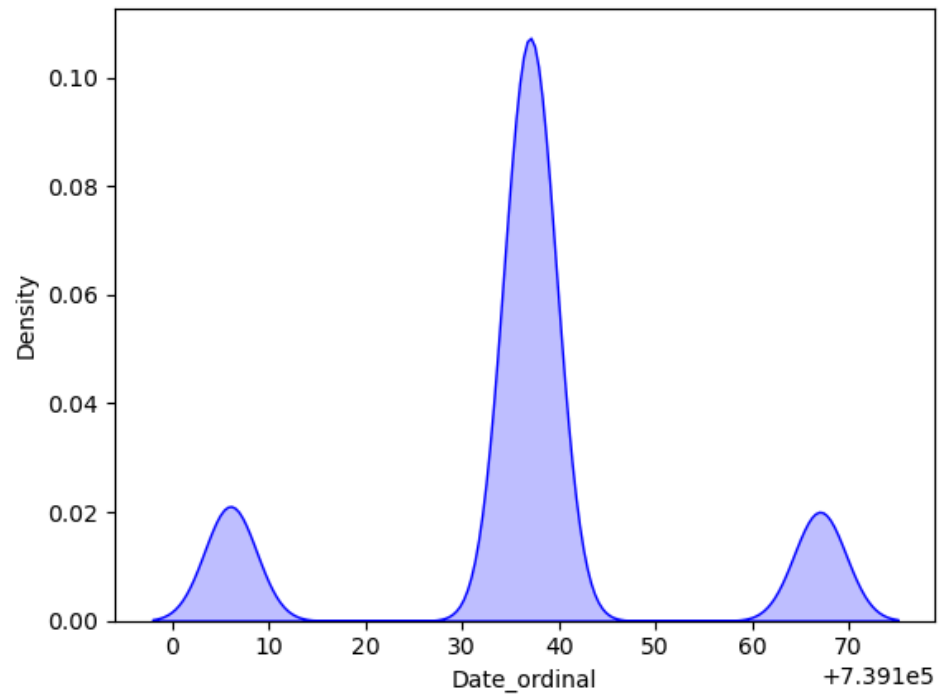
○



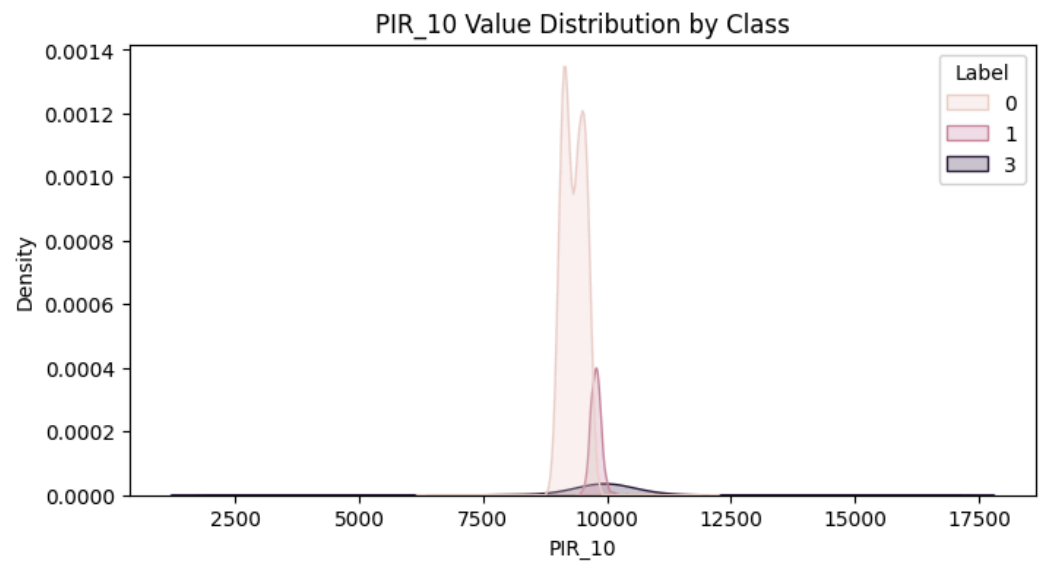
○

	Date	Count
■	2024-09-08	5532
■	2024-08-08	1086

■ 2024-10-08 1033



○



3. Feature Engineering and Data Preprocessing

- **Pre-processing choices:**
 - **SMOTE and dimensionality reduction techniques** (like PCA) are generally not applied in this pipeline because the **data is time-sequenced and structured for LSTM-based models**. Applying SMOTE would disrupt the temporal coherence between consecutive timesteps, while techniques like PCA may distort feature interpretability and inter-timestep relationships, which are crucial for sequential learning.
 - **PIR Sensor Data Processing**
 - The 55 PIR sensor columns (identified automatically by selecting columns that start with "PIR_") are scaled to a range between 0 and 1 using `MinMaxScaler`. This normalization helps make the sensor readings consistent and easier for deep learning models to learn from.
 - **Metadata Processing:**
 - **Standardizing Continuous Metadata:**

The Temperature is scaled using a `StandardScaler` (renamed here as `temp_scaler`), and continuous time features (e.g., cyclical representations of hour, minute, and second, and the binary `is_weekend` flag) are also maintained as continuous variables.
 - **Combining Metadata:**
 - All metadata features used for modeling are assembled as follows:
 - Categorical features: Dummy variables created from `dayofweek` (`dow_` columns)
 - These combined columns form the metadata input to the model.
 - **Label Encoding:**
 - The activity labels, originally in string format, are converted into numerical values using `LabelEncoder`. After that, they are one-hot encoded using `to_categorical()` to prepare them for multi-class classification.
 - **Creating Input Data for the Model**
 - **PIR Sensor Data:**
 - The sequential PIR readings are reshaped to fit the input format expected by time-series models.
 - **Metadata:**
 - The non-sequential metadata is kept as a 2D array after scaling, ready to be passed into the model alongside the PIR data.
-

4. Model and Architecture Choices and Hyperparameter tuning:

- **Why not MLP?**

MLPs treat all inputs as independent and ignore the order of data, making them unsuitable for sequential tasks like PIR signals. They can't capture temporal patterns or retain past context like LSTM/GRU can. CNNs also perform better at spotting local time-based patterns. So, MLPs are limited for this use case.

- **Why not transformers?**

1. Size of the Dataset

As the dataset is relatively small, transformers might overfit quickly or not learn effectively (they need large data).

2. Sequence Length and Simplicity of Temporal Patterns

As seen in the correlation matrix, the columns nearby are highly related (about 5-7 adjacent) so we can say that the long-range dependencies are less and also the sequences are short, hence CNNs or LSTM or GRUs can be more efficient and would be less computationally intensive. Hence transformers might be an overkill.

❖ **Late fusion (Option 1)**

Combining Sequential and Non-Sequential Data (choice of hybrid architecture)

In our hybrid architecture, we separate the **sequential** (e.g., PIR sensor time-series) and **non-sequential** (e.g., time-based metadata like hour, day, temperature) inputs.

The sequential data is first passed through a temporal model such as **LSTM, CNN, or GRU**, which is designed to capture time-dependent patterns and trends. The output of this model—typically a compressed, high-level representation of the temporal dynamics—is then **concatenated** with the non-sequential features.

This combined representation is then fed into a **dense neural network**, which is responsible for modeling complex relationships between the learned temporal features and the additional static/contextual features.

The reason for this approach is twofold:

1. **Specialization of Networks:** Sequential models are better suited for time-series dependencies, while dense networks excel at capturing interactions among static features.

2. **Preservation of Structure:** Keeping the sequence structure separate avoids distortion from flattening or concatenating time-dependent and independent data prematurely, which could otherwise confuse the model and reduce performance.

This strategy enables the model to leverage the strengths of both types of features and learn richer, more context-aware representations, ultimately improving classification performance.

CNN:

In our 1D CNN architecture, we empirically selected a kernel size in the range of 5 to 7 based on observed feature correlations. Since CNNs are highly effective at capturing **local patterns**, smaller kernel sizes allow the model to focus on short-term dependencies or localized activations in the sensor data. This aligns well with the **temporal locality** in PIR sensor signals.

Using a **larger kernel size**, on the other hand, comes with trade-offs like diluted local-sensitivity, increased parameters leading to overfitting.

Therefore, we found kernel sizes between 5–7 to be optimal for our use case, balancing model capacity with the ability to effectively capture meaningful local correlations in the PIR signal.

LSTM:

By increasing the **patience** from 5 to 10 and the **learning rate** from 0.001 to 0.005 in the Adam optimizer, we observed a sharp accuracy jump from **78% to 97%**, along with a notable spike in **F1 score**. This improvement came with minimal hyperparameter tuning, showing the model's high sensitivity to these changes.

GRU:

We observed performance improvements by increasing the **learning rate** to **0.005**, allowing faster convergence. The **GRU layer sizes** (64 and 32 units) were selected to balance expressiveness and efficiency. A **dropout rate of 0.3** was used to prevent overfitting, and **batch normalization** was applied to stabilize training. The model showed strong performance without extensive hyperparameter tuning, indicating a robust initial setup.

Reasons for Choosing LSTM:

1. **Excels at Capturing Long-Term Dependencies:**
LSTM is specifically designed to retain information over longer sequences, making it ideal for time-series data with temporal context.

2. **Better Memory Control:**

With its gating mechanisms (input, forget, and output gates), LSTM can selectively retain or discard information, leading to more effective learning in sequential tasks.

3. **More Parameters = Greater Flexibility:**

Although heavier, the larger number of parameters in LSTM allows it to learn complex temporal dynamics more thoroughly than GRU or CNN.

4. **Tailored for Sequential Data:**

Unlike CNNs that focus on local patterns, LSTMs are inherently built to handle ordered data and preserve time-based relationships.

5. **Empirically Proven:**

In our case, LSTM outperformed both CNN and GRU, showing a significant jump in accuracy and F1-score after minor tuning, highlighting its suitability for the dataset.

Option2 - Early fusion:

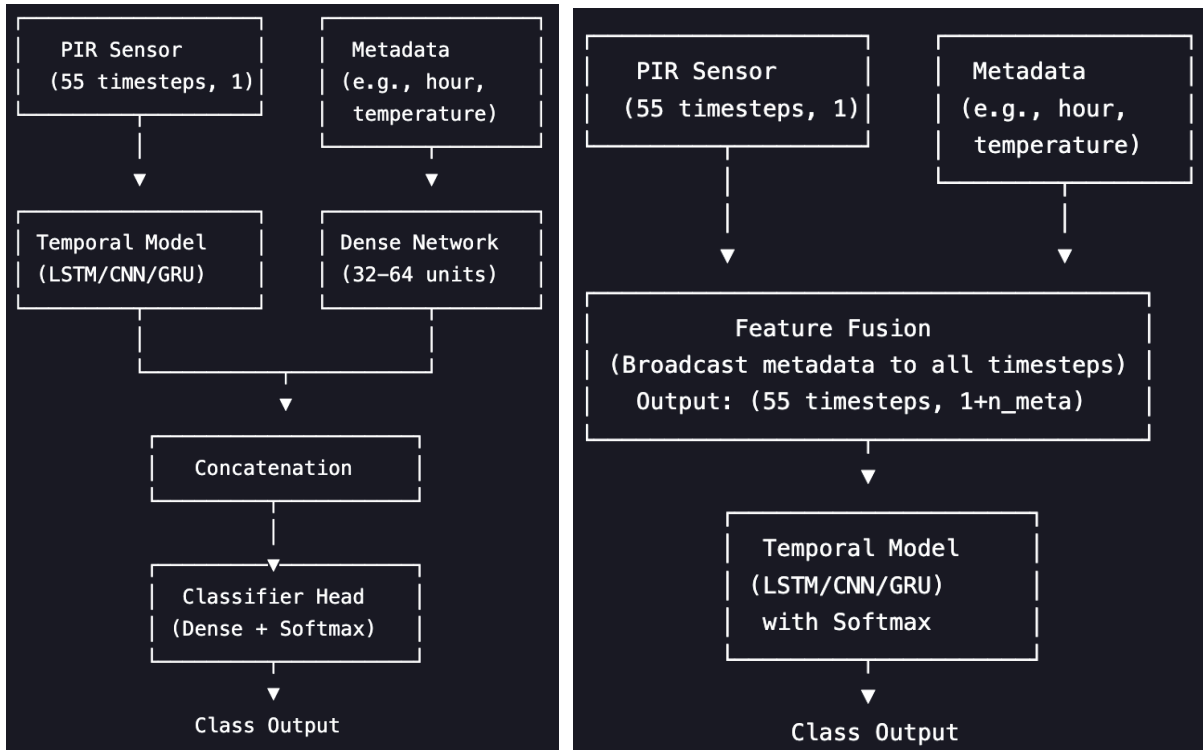
Early fusion involves combining both sequential (PIR sensor) and non-sequential (time-based metadata) features at the input level. In this approach, metadata is broadcast across all timesteps and concatenated with PIR values before being fed into the model. This allows the LSTM to learn temporal patterns in context with auxiliary information from the start.

Architecture: The model uses a two-layer LSTM network with 64 and 32 units respectively, followed by batch normalization, a fully connected dense layer, and dropout for regularization. The final softmax layer outputs class probabilities.

Choice of late fusion over early fusion:

- Useful when sequential and non-sequential features are **not strongly correlated which can be seen from correlation matrix**.

FlowChart for better understanding: (Option 1, Option 2 respectively)



5. Model Selection & Validation:

→ **Hybrid Model Architecture:** The model is designed with two separate input branches that merge for final classification:

- **Sequential Branch (PIR Sensor Data):**
 - Two LSTM layers (64 units then 32 units) extract temporal features from the sequential PIR readings.
 - BatchNormalization is applied after LSTM layers to stabilize and improve the training process.
- **Metadata Branch (Non-Sequential Data):**
 - A Dense layer (32 units, ReLU activation) processes the metadata features, followed by BatchNormalization.
- **Feature Fusion and Classification:**
 - The outputs from both branches are concatenated.
 - A subsequent Dense layer (64 units, ReLU activation) refines the fused features.

- Dropout (0.3) is introduced to reduce overfitting.
- The final Dense layer uses softmax activation to output class probabilities for each activity label.

→ Model Compilation

The model is compiled with the Adam optimizer (learning rate set to 0.005) and uses categorical cross entropy as the loss function. Accuracy is the primary performance metric.

→ 5-Fold Stratified Cross-Validation

- The code performs 5-fold stratified cross-validation using `StratifiedKFold`.
 - For each fold:
 - The dataset is split into training and validation sets while preserving the class distributions.
 - The hybrid model is built and trained on the training data, incorporating class weights to **manage class imbalance**.
 - `EarlyStopping` is used (monitoring `val_accuracy`, with a patience of 10) to restore the best weights if the model performance does not improve.
 - Training history (loss and accuracy curves) is plotted for each fold.
 - At the end of each fold, predictions are made on the validation set, and metrics such as validation accuracy, weighted F1, and macro F1 scores are computed and recorded.
 - A classification report is printed for each fold.
 - Finally, mean and standard deviation of accuracy and F1 scores across all folds are printed, summarizing cross-validation performance.
-

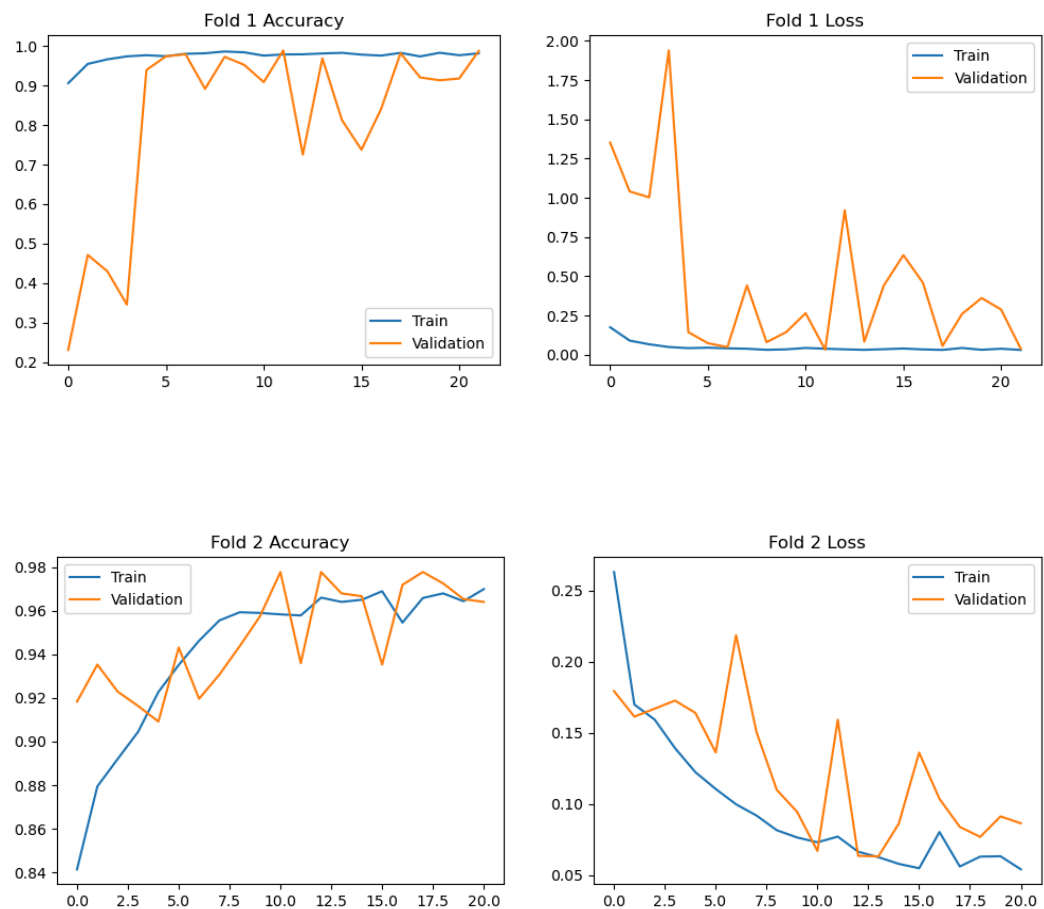
6. Test Data Evaluation

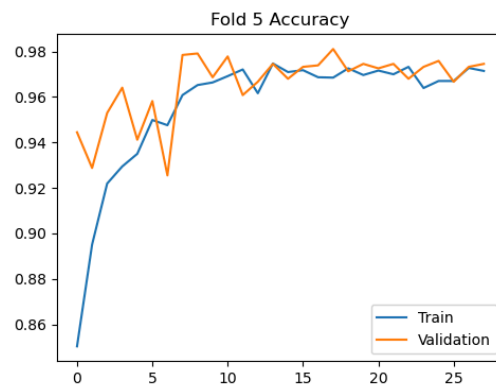
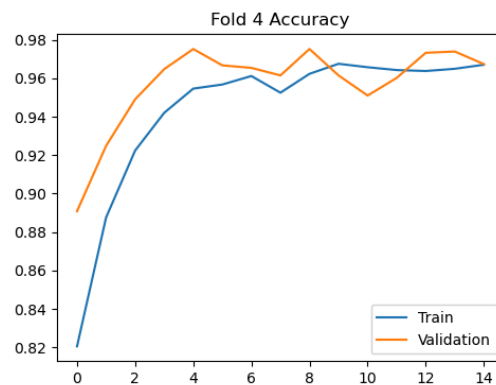
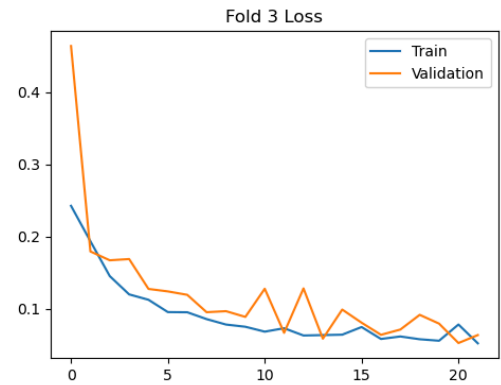
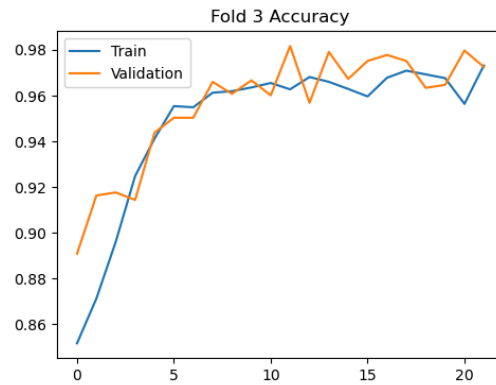
- The test dataset (`pirvision_office_dataset2.csv`) is processed similarly to the training data.
- Date and time processing are applied.
- One-hot encoding is ensured to match the training dimensions.
- The same scalers are used for normalizing PIR readings and standardizing the `Temperature_F` feature.

- Predictions are made on the test data by reshaping the PIR data and preparing metadata as a 2D array.
 - The classification report is printed on the test data.
 - We print the true and predicted labels for a sample of 10 test instances.
-

7. Visualizations:

Training vs. Validation Loss Curves and Training vs. Validation Accuracy Graphs(For all folds) : The x-axis represents the number of training epochs.

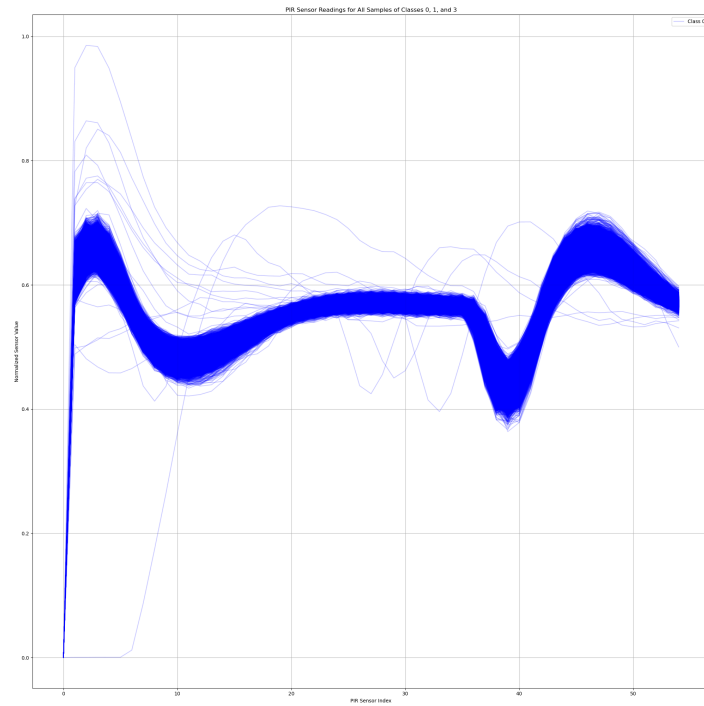




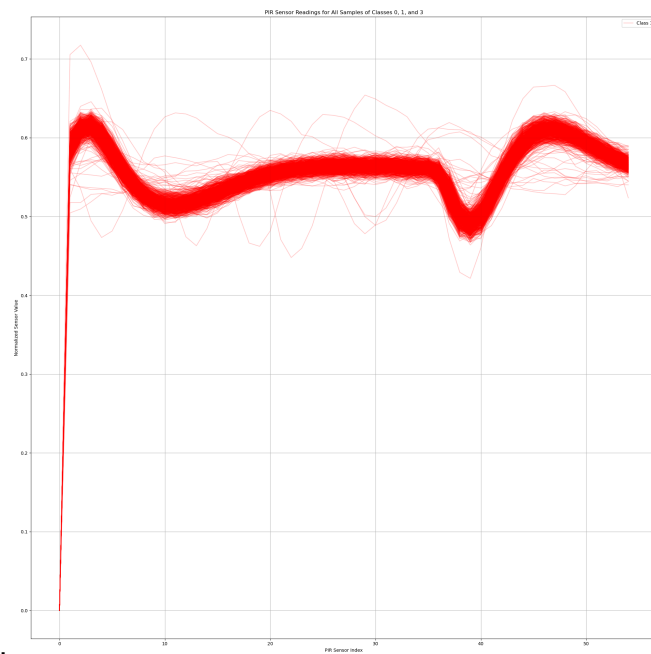
→ **Additional Visualizations are provided in our notebook:**

- **Combined Visualization of PIR Sensor Readings:**
A single plot overlays PIR sensor readings for all samples across three selected classes (0, 1, and 3). This plot uses different colors (blue for Class 0, red for Class 1, and yellow for Class 3) with transparency (alpha=0.2) to show how the sensor readings vary across the classes.
- **Individual Class Visualizations:**
Separate plots are generated for:

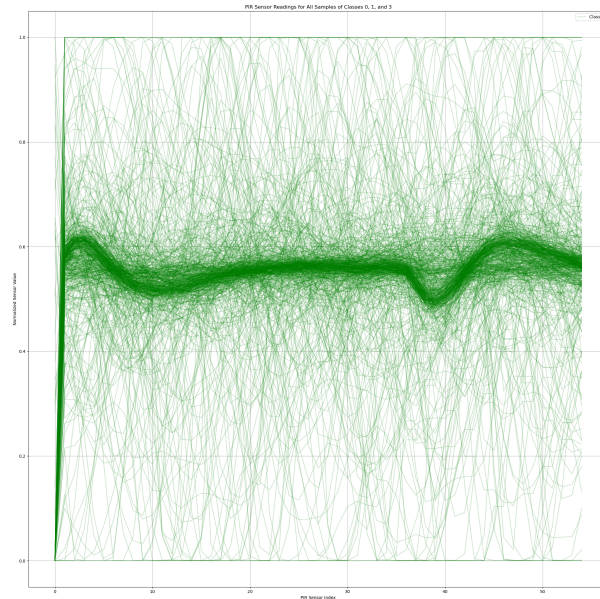
- Class 0: Displays PIR sensor readings for all samples belonging to Class 0.



- Class 1: Displays PIR sensor readings for all samples belonging to Class 1



- Class 3: Displays PIR sensor readings for all samples belonging to Class 3.



8. Results & Insights:

- **Result:**

-

Final Cross-Validation Results:

Mean Accuracy: 0.9732 (± 0.0081)

Mean Weighted F1: 0.9814 (± 0.0044)

Mean Macro F1: 0.9684 (± 0.0074)

Total time taken to train (all folds): 380.14371691700217):

- **Class-wise Precision, Recall, Confusion Matrix** are printed in our notebook
- **Insights:** Class 3 always gave precision, recall, f1 score of 1, this could be due to the fact that its distribution is quite different from class 0 and 1, hence could be easier for the models to distinguish it from other classes making it always correctly classified. This can be seen in class-wise visualization (3 is randomly distributed across space while 1 and 0 are following specific spatiality(following peaks)).
- **Results Summary**
 - The 5-fold cross-validation results show that the hybrid model consistently achieves high accuracy and low loss across all folds

- A high mean accuracy of 97.32% with a low standard deviation ($\pm 0.81\%$) indicates consistent overall predictions.
 - The weighted F1 score of 98.14% ($\pm 0.44\%$) reflects handling of class imbalance, while the macro F1 score of 96.84% ($\pm 0.74\%$) shows that the model performs well across all classes.
 - **Model Performance Analysis:**
 - Overall, the model demonstrates strong performance on the dataset. The **hybrid approach**, which processes sequential PIR sensor data and non-sequential metadata separately before fusing them, appears to help the model effectively learn both temporal and contextual features.
 - Before, we used a simple MLP model for both the sensor data and the metadata combined, not treating them as sequential. That gave us a lower accuracy of about 92–93%. This happened because the model couldn't understand the time-based changes in the sensor data properly.
 - But when we used a hybrid model that handled sensor data and metadata separately—treating the sensor data as a sequence—the accuracy improved to around 98%.
 - This shows that using hybrid methods for this particular data helps the model learn better and make more accurate predictions.
-

9. Challenges & Future Improvements:

Future improvements may include:

- Experimenting with alternative sequence models and their combinations (combination of LSTM, GRU or CNN-based architectures).
- Refining preprocessing, such as incorporating advanced outlier detection methods.
- Exploring additional feature engineering techniques.
- Evaluating the approach on extended or complementary datasets.